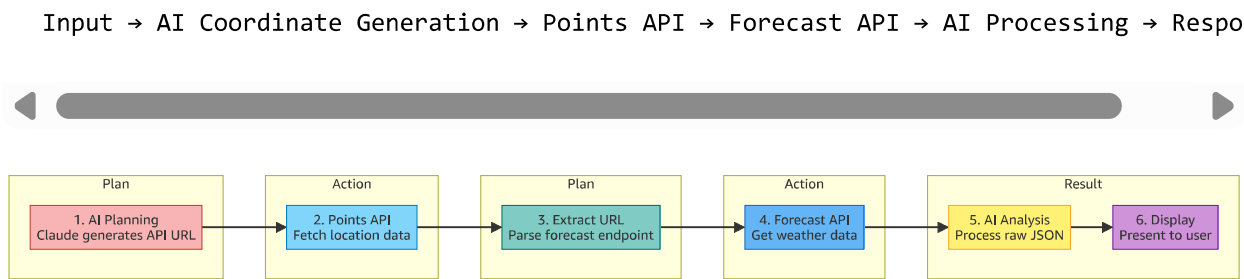


Understanding Our Agent Architecture

The Six-Step Workflow

In section 1 we introduced the four building blocks. Here is how they chain together to handle one weather request:



#	Step	Phase	What runs
1	AI Planning	Plan	Claude generates the Points API URL
2	Points API	Action	cur1 fetches NWS location metadata
3	Extract URL	Plan	Parse the Points response to get the forecast URL
4	Forecast API	Action	cur1 fetches the actual weather data
5	AI Analysis	Result	Claude turns raw JSON into a readable summary
6	Display	Result	Present the answer to the user

Steps 1, 3, and 5 are the "thinking" steps. Steps 2 and 4 are the "doing" steps. Step 6 is presentation. Step 3 is a deterministic JSON parse — no model call needed.

How the National Weather Service APIs Work

The NWS provides free weather data through a two-step API process. The first call tells you where to make the second call.

- **Points API** — <https://api.weather.gov/points/{lat},{lon}>
 - Takes latitude and longitude coordinates as input
 - Returns which NWS forecast office covers that location and the grid coordinates within it
 - Example: Seattle (47.6062, -122.3321) → SEW office, grid 124,67
- **Forecast API** — <https://api.weather.gov/gridpoints/{office}/{gridX},{gridY}/forecast>
 - Uses the office and grid coordinates from the Points API
 - Returns the detailed forecast as JSON
 - Example: `gridpoints/SEW/124,67/forecast` returns Seattle's forecast

The NWS divides the US into a grid where each forecast office covers specific squares, which is why two calls are needed: the first resolves a location to a grid square, the second reads the forecast for that square.

The Complete Sequence

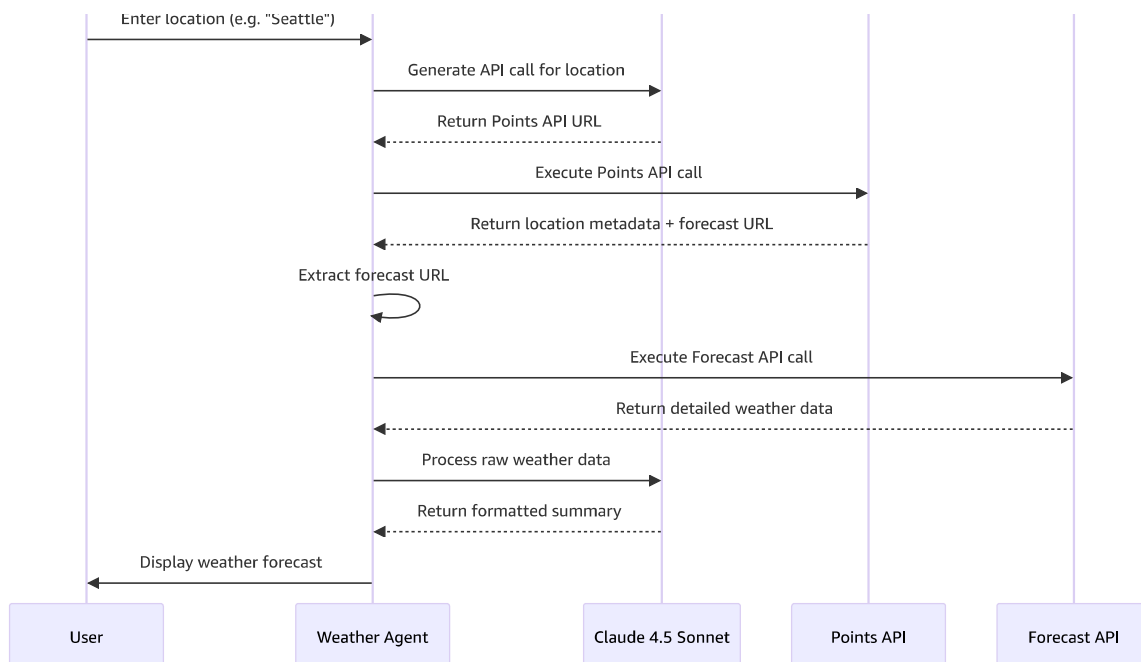
Here is how the user, the agent, Claude, and the two NWS APIs interact end to end:

© 2008 - 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

[Privacy policy](#)

[Terms of use](#)

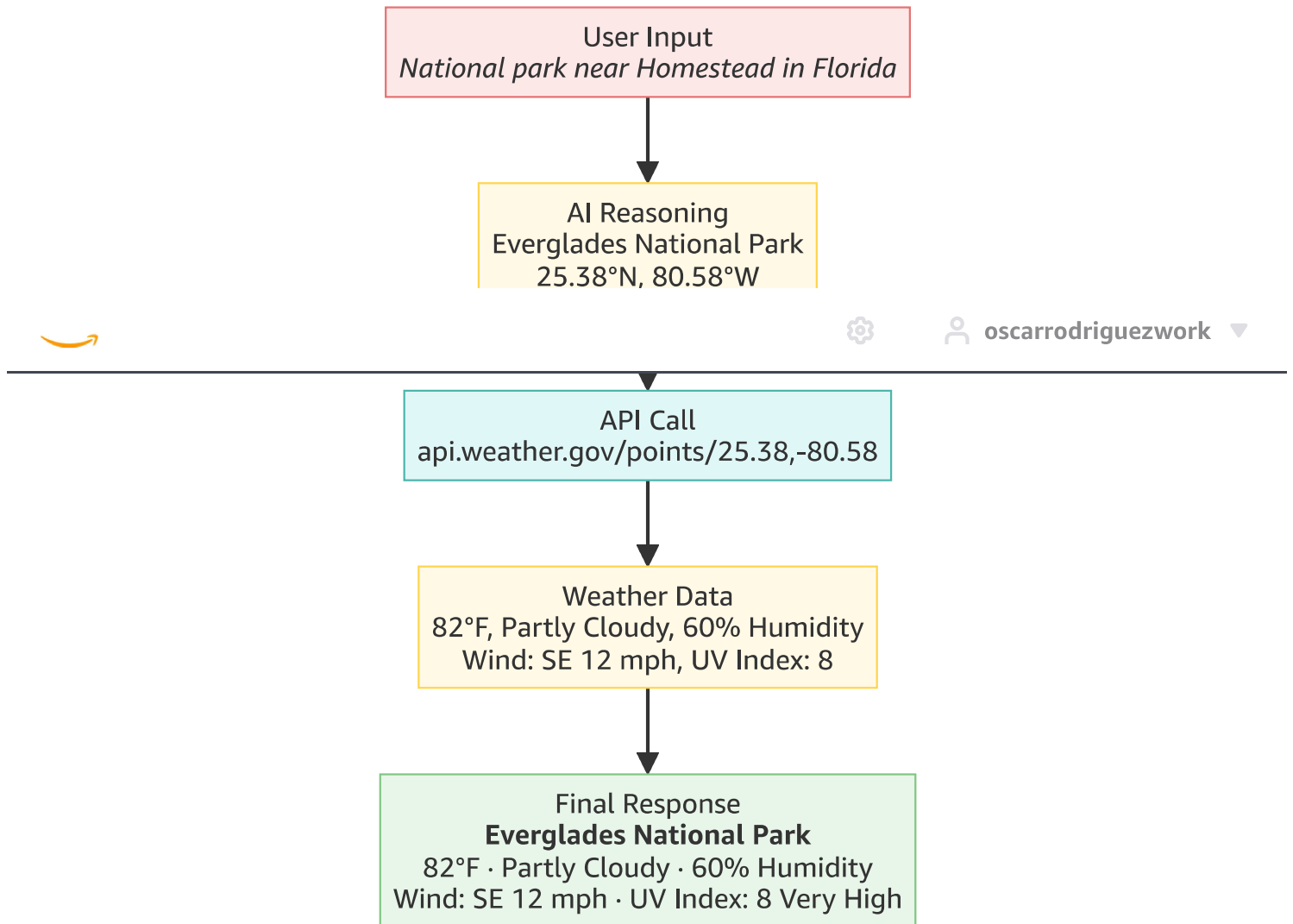
[Cookie preferences](#)



The Weather Agent is the conductor — every message flows through it. Claude is consulted twice (once to plan, once to analyse), and each NWS API is called once.

From Natural Language to Forecast

The interesting part of agentic AI shows up when the user gives a vague description instead of an exact address. Below, "National park near Homestead in Florida" becomes coordinates for the Everglades, then a real forecast:



The user never typed "Everglades" or 25.38, -80.58. Claude inferred both from the description, and the agent walked the rest of the way deterministically.

Implementation Notes

A few things worth knowing before we start writing code:

- **The planning prompt does the heavy lifting.** Claude's ability to take "downtown Portland" or "90210" and emit a correct `https://api.weather.gov/points/{lat},{lon}` URL is what makes this agent feel intelligent. We will look at the prompt in section 4.
- **The forecast URL is extracted, not generated.** The Points API response includes the forecast URL directly under `properties.forecast`. Asking the model to construct it would be slower and less reliable, so we just parse the JSON.
- **The analysis prompt is where formatting lives.** The same forecast JSON can become a one-line summary or a multi-day outlook depending on how we ask. We will keep the prompt practical and readable rather than exhaustive.

What Makes This "Agentic"

A traditional implementation would hardcode every supported location. The agentic version lets the model figure it out:

```
1  # Traditional: Hardcoded and brittle
2  if location == "Seattle":
3      api_url = "https://api.weather.gov/gridpoints/SEW/124,67/forecast"
4
5  # Agentic: AI figures it out
6  ai_prompt = f"Generate NWS API calls for: {location}"
7  api_urls = claude_4_sonnet(ai_prompt)
```



The model reasons about the input, plans the call, and adapts to inputs we never anticipated. That is the whole idea.

[Previous](#)[Next](#)